PES UNIVERSITY

ORANGE PROBLEM

NAME: Pavankumar H

SRN: PES2UG24CS345

SUB: OPERATING SYSTEM

The test program verifies:

- Blob storage and retrieval (write, read back, compare)
- Deduplication (same content → same hash → stored once)
- Integrity checking (detects corrupted objects)

Screenshot 1A: Output of ./test_objects showing all tests passing.

```
pavankumarh-345@LAPTOP-89VUF1P0:~//test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c744re9bbe2c4b82fc1
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7dd413eab
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.

pavankumarh-345@LAPTOP-89VUF1P0:~/os-u4-orange-problem$
```

Screenshot 1B: find .pes/objects -type f showing the sharded directory structure.

```
pavankumarh-345@LAPTOP-89VUF1P0:~/os-u4-orange-problem$ find .pes/objects -type
f
.pes/objects/2a/594d3922787fbaeb7287418aec99c8fcecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5be0669bb5cfa285c7d213aa09227ed0221bbe9d5e5c4b9aafc12

.pes/objects/d5/8213f5be0669bb5cfac7d213aaa09227ed0221bbe9d5e5c4b9aafc12
.pes/objects/25/ef1fa07ea6552800dc8075e6b7ae34b337afed9b46alafe1lec4f476
```

The test program verifies:

- Serialize → parse roundtrip preserves entries, modes, and hashes • Deterministic serialization (same entries in any order → identical output) Screenshot 2A: Output of ./test_tree showing all tests passing.

```
pavankumarh-345@LAPTOP-89VUF1P0:~/os-u4-orange-problem$
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.

pavankumarh-345@LAPTOP-89VUF1P0:~/os-u4-orange-problem$
```

Screenshot 2B: Pick a tree object from find .pes/objects -type f and run xxd .pes/objects/XX/YYY... | head -20 to show the raw binary format.

```
pavankumarh-345@LAPTOP-89VUF1P0:~/os-u4-orange-problem$
.pes:
 objects refs

.pes/objects:

.pes/refs:
 heads

.pes/refs/heads:

pavankumarh-345@LAPTOP-89VUF1P0:~/os-u4-orange-problem$
```

Screenshot 3A: Run ./pes init, ./pes add file1.txt file2.txt, ./pes status — show the output.

```
pavankumarh-345@LAPTOP-89VUF1P0:~/os-u4-orange-problem$ echo "hello" > file1.txt
pavankumarh-345@LAPTOP-89VUF1P0:~/os-u4-orange-problem$ .pes add file1.txt file22.txt
pavankumarh-345@LAPTOP-89VUF1P0:~/os-u4-orange-problem$ .pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  output.txt
  untracked:  a.txt
  untracked:  commit.c
  untracked:  tree.c
  untracked:  test_tree
  untracked:  README.md
  untracked:  pes/objects
  untracked:  Makefile
  untracked:  test_tree.c
  untracked:  commit.h
  untracked:  test.h
  untracked:  test_sequence.sh
  untracked:  objects.h
  untracked:  objects.h
  untracked:  objects.c
  untracked:  test_tree
  untracked:  objects.h
  untracked:  objects.c
  untracked:  test_objets
pavankumarh-345@LAPTOP-89v~/os-u4-orange-problem$
```

Screenshot 3B: cat .pes/index showing the text-format index with your entries.

```
pavankumarh-345@LAPTOP-89VUF1P0:~/os-u4-orange-problem$ cat .pes/
100644 2cfd83d9ee29543b3aa8777421fdec7e3fa183d337769025de57dbbeb4
100644 e00c50e16a2df88f8d6bf809e181ad028da6e6719f359f7e65df60199f7
```

Screenshot 4A: Output of ./pes log showing three commits with hashes, authors, timestamps, and messages.

```
Committed
commit 1776263263
author sham_PES2UG24CS455
time 1776263263
first commit
```

Screenshot 4B: find .pes -type f | sort showing object store growth after three commits.

```
Committed
commit 1776263503
author sham_PES2UG24CS455
time 1776263503
second commit
```

Screenshot 4C:cat .pes/refs/heads/main and cat .pes/HEAD showing the reference chain.

```
pavankumarh-345@pavankumarh-345-VMware-Virtual-Platform:~/os-u4-orange-pr
.pes/refs/heads/main
  9a8b7cd65e4f3g2h1iojj0j30b663ff9aacf8229aaaec7739929aadc8412345
  9a8b7cd65e4f3g2h1iojj0j30b663ff9aacf8229aaaec7739929aadc8412345
  9a8b7cd65e4f3g2h1iojj0j30b663ff9aacf8229aaaec7739929aadc8412345
  9a8b7cd65e4f3g2h1iojj0j30b663ff9aacf8229aaaec7739929aadc8412345
  9a8b7cd65e4f3g2h1iojj0j30b663ff9aacf8229dddd6a24cm02,j:177625099$}
pavankumarh-345@pavankumarh-345-VMware-Virtual-Platform:~/os-u4-orange-probl
pavankumarh-345@pavankumarh-
 rein
 ref: refs/heads/main

pavankumarh-345@pavankumarh-345-VMware-Virtual-Platform:~/os-u4-orange-proble
rein
ref:  refs/heads/main

pavankumarh-345@pavankumarh-345-VMware-Virtual-Platform:~/os-u4-orange-problem
pavankumarh-345@pavan-
 rein
 ref: refs/heads/main
```

Q5.1: Branch & Checkout

A branch in Git is a file inside .pes/refs/heads/ that stores a commit hash.

To implement pes checkout <branch>:

Update .pes/HEAD to: ref:

refs/heads/<branch>

Read commit hash from .pes/refs/heads/<branch>

Load the corresponding tree from object store Update

working directory to match that tree:

Create/update files

Delete extra files

Why complex?

Must overwrite working directory safely

Must handle uncommitted changes

Must ensure consistency between index, working directory, and object store

If file di ers in both → conflict → refuse checkout

Key idea:

Working Dir ≠ Index → modified

Index ≠ Target Tree → branch change

Both true → conflict

Q5.3: Detached HEAD

Detached HEAD means:

HEAD → commit (not a branch)

What happens:

New commits are created

But NOT attached to any branch

They can get lost (unreachable)

Recovery:

Create a branch pointing to that commit:

pes branch <name>

Or manually update ref

Q6.1: Garbage Collection

Goal: Remove unreachable objects

Algorithm:

Start from all branch heads (refs/heads/*)

Traverse: commit → parent commit → tree

tree → blobs/trees Mark all visited objects

Data structure: Hash Set (visited set) →

O(1) lookup Delete:

Remove objects NOT in visited set

Estimation:

~100,000 commits

Each commit → 1 tree + blobs

~200,000–300,000 objects traversal

Q6.2: GC Race Condition

Problem:

GC runs while commit is happening Scenario:

Commit creates new object (not yet referenced)

GC runs → sees object as unreachable

GC deletes it

Commit updates HEAD → now points to missing object

Why dangerous:

Leads to corrupted repository

How Git avoids this:

Uses locking

Uses atomic reference updates

GC only runs when repo is stable

Uses temporary files before final write